智视灵动机械臂系统

——芯语智分 基于 Zynq 软硬件协同的语音视觉垃圾分拣系统

目录

摘 要. 3

第一部分　作品概述 4

1.1　选题背景与前提 4

1.2　功能与特性 4

1.3　应用领域 5

1.4　主要技术特点 6

1.5　主要性能指标 6

1.6　主要创新点 7

1.7　设计流程 8

第二部分　系统组成及功能说明 9

2.1　整体介绍 9

2.2　硬件系统介绍 10

2.2.1　硬件整体介绍 10

2.2.2　机械设计介绍 11

2.2.3　电路各模块介绍 13

2.3　软件系统介绍 14

2.3.1 软件整体介绍.....14
2.3.2 软件各模块介绍.....15
2.4 关键算法与创新设计.....25
2.4.1 从“识别”到“允许动”：候选门控的可核对判据.....25
2.4.2 在途信用与整帧事务：让定点流水在背压下不串帧.....26
2.4.3 面向资源的定点设计：位宽由最坏边界反推.....26
2.4.4 单清单寄存器与生产端语音边界：让系统可被复核.....26
2.5 软硬件划分依据与接口设计.....27
第三部分 完成情况及性能参数.....29
3.1 整体介绍.....29
3.2 工程成果.....30
3.2.1 机械成果.....30
3.2.2 电路成果.....31
3.2.3 软件成果.....31
3.3 特性成果.....32
3.3.1 优化过程与前后对比.....33
3.3.2 资源与时序.....35

3.3.3 测试与验证记录.....36
3.4 大模型协作记录.....37
3.5 技能包的提炼过程.....38
3.6 从零复现说明.....39
3.6.1 准备环境.....39
3.6.2 取源码并固定版本.....40
3.6.3 复现固定工位演示.....40
3.6.4 构建四功能工程与传输库.....40
3.6.5 模型入册与私有配置.....41
3.6.6 启动与保存证据.....41
第四部分 总结.....43
4.1 后续发展方向.....43
4.2 心得体会.....43
第五部分 参考文献.....45

摘 要

生活垃圾从投放点到资源化或焚烧处置之间，垃圾站的分拣是决定后序垃圾回收以及垃圾焚烧能否成功的关键一环，垃圾站分拣的痛点在于：来料混杂，靠人一件件拣，粉尘与尖锐物多。芯语智分（Xinyu SmartSort）把“听见指令—看清目标—落到行动”做成一个可复现的能够独立实现垃圾分拣的取代单元：首先说“你好小亚”唤醒，再说“开始垃圾分拣”，系统准备观察姿态，在连续稳定帧上确认目标，播报物品与类别，由 DOFBOT SE 机械臂完成固定工位抓取投放，把重复拾取与直接接触的机会交给机器。

系统以 ALINX Z19 为平台，按“确定的算术下沉到 PL、语义与接口保留在 PS”划分软硬件。PL 承担逐像素的 RGB 校正、HSV 掩码、ROI 计数与包围框、64 位一阶矩统计，逐样本的直流抑制、63 抽头 FIR 与软噪声门，并把候选复核实现为可综合的时空门控；PS 承担 V4L2 采集解码、CPU YOLO 检测、坐标解释、Vosk 受限口令识别与有界任务服务。PL 做像素级校正、掩码和统计，PS 跑 YOLO 与坐标，再由 PL 判断这次能不能抓——同帧唯一目标、面积落区间、连续三帧严格更新、四边界相对首帧锚点抖动不超容差、一次显式消费给许可。再加上整帧事务（首像素冻结、末像素闭合、坏帧不发布）与语音边界（每轮重建采音—滤波—识别）。

四功能集成工程已在 Vivado 2020.1 下完成实现与 bitstream 生成：100 MHz 下 WNS +0.632 ns，等效 Fmax 约 106.7 MHz，比目标高约 6.7%；占用 7,292 LUT、14 DSP（占器件 1.40%、0.71%），估算功耗 4.167 W。与原 hd 相比，板端发布率由 1.924 fps 升到 7.26 fps（3.8 倍），采集至发布由 567.99 ms 降到约 187 ms（降 67%），PIO 处理由 472.64 ms 降到 116.78 ms（降 75%），FIR 由 63 个乘法器并行改为 1 个时分复用。它的意义是把重复拾取与直接接触交给机器，并让每次分类留下可统计的记录——回收纯度、湿垃圾分出量以及有害垃圾拦截量。

关键词：垃圾分拣；Zynq PS/PL 协同；定点流处理；候选门控；语音交互；机械臂

第一部分　作品概述

1.1　选题背景与前提

垃圾分类的标准已经很细，难在垃圾站的分拣位：来料混杂，靠人一件件拣，粉尘与尖锐物多，长期人工接触存在职业健康风险，而分拣质量又直接影响后续三条去向：可回收物能否进入再生利用、湿垃圾能否被分出以提高入炉热值、有害垃圾会不会污染整批可回收物，因此垃圾站的分拣至关重要。

现有的基础中，工业分选线（TOMRA AUTOSORT、AMP、ZenRobotics）用传感识别与机械分选处理后端连续大流量物料，解决的是规模化问题，难以做到实时记录；桌面教育套件（如 Yahboom DOFBOT 配套工程）解决的是“演示一次识别”，模型与动作可用，但任务编排、接口一致性与过程记录往往不成体系。两者之间缺一个能完成一次完整任务、又能被复制到不同点位的节点以及一条把“识别到目标”变成“允许执行”的判据。

作品定位在这个缺口：不重造检测模型与机械臂，把工作放在任务编排、PS/PL 划分、候选门控、可观测性与可复现性上。与工业线的差别在应用层次——单件、固定拾取位、四类去向，而不是连续物料处理；与教育套件的差别

在工程闭环——一次任务的状态、帧身份、许可与动作阶段都能被记录和复核。

1.2 功能与特性

一次任务由六个动作组成：唤醒、启动、观察准备、稳定识别、类别播报、抓取投放；任务结束后回到等待唤醒，下一轮必须重新唤醒。窗口同步显示现场画面、识别结果、任务阶段、语音与任务记录，以及可回收物、有害垃圾、湿垃圾与干垃圾四个投放区的当前状态。

系统提供四组可独立阅读与复用的能力。视频预处理完成 RGB 通道定点校正、HSV 阈值掩码、ROI 计数、包围框与一阶矩统计，参数按帧冻结；垃圾识别完成真实 YOLO 推理、模型清单核对与类别到业务分组的映射，未知标签显式报错；视觉定位分别记录检测框中心、颜色质心与平面毫米坐标三种语义语音前端把 USB PCM 经直流抑制、63 抽头 FIR 与可选软噪声门后送入受限口令识别，串口语音模块作为另一路事件源保留。

三类特性贯穿上述功能：确定性，同一帧用同一组参数且统计覆盖到末像素；可追溯，每个许可与结果都带会话、帧号与来源；可配置，阈值、ROI、面积区间、抖动容差与口令词表都由配置给出。

1.3 应用领域

作品的主场景是垃圾站（中转站、集散分拣点）的来料分拣。混合生活垃圾经破袋、摊铺与单件化后进入工位：相机识别目标，候选规则判断“这次能不能动”，机械臂按类别投放，四个容器对应可回收物、有害垃圾、湿垃圾与干垃圾（图 1）。这是后续处置的基础——分出可回收物才能进入再生利用，纯度越高价值越大；分出湿垃圾（厨余）可降低入炉物料含水率、提高低位热值；隔离有害垃圾则避免废电池渗液等污染整批可回收物。分拣现场是重复搬运、粉尘与尖锐物并存的岗位，把识别与拾取交给工位，可减少人工直接接触对有害垃圾尤其有意义。

同一套任务、接口与证据可复现成节点：园区、写字楼、社区与展馆的投放点只需更换样本、模型、容器与拾取位置；多节点回传任务量、类别与失败原因，可形成运营与碳减排看板。

面向高校，工作站也是嵌入式与机器人实践的教学载体：同一任务串联像素处理、模型调用、候选规则与执行接口；小型回收服务点可复用同一接口，按物料重新选择模型、夹具与容器后独立验收。

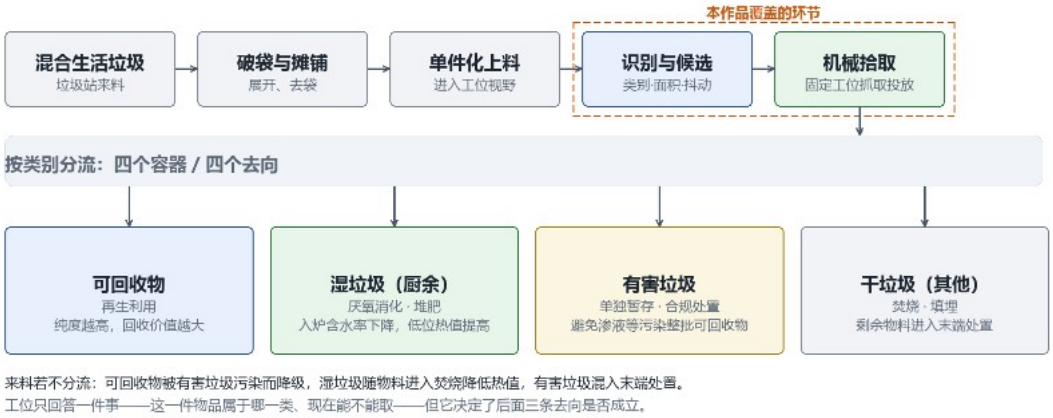


图 1　垃圾站分拣工艺与本作品的位置

1.4　主要技术特点

（1）定点流水与在途信用。像素与样本通路只在握手成功时推进；对没有 ready 的自由运行算术核，按“已占用＋已预留”核算 FIFO 容量，未产生的结果也计入压力，下游停顿既不溢出也不串帧。

（2）整帧事务。配置经 staging → commit → pending → active 生效，首像素即使用本帧参数；统计在末像素闭合后冻结，主机 ACK 才接纳下一帧，半帧与跨帧参数不会混用。

（3）语义候选与固定锚点。复核以同帧检测框的区域与面积为准；候选段以首帧四边界为锚点，配合严格新帧与一次消费。

（4）语音边界。任务开始与结束重建采音、滤波与识别代际，忙碌期间的

口令不会补成下一轮唤醒。

（5）单一寄存器清单。偏移、位字段与设备 ID 只在 registers.json 维护，RTL、C 与 Python 为其投影，三端不会各解一位。

（6）控制与展示分离。控制进程唯一持有相机、语音与机械臂，展示线程旁路复制最新帧，窗口只读，不提供触发运动的入口。

1.5 主要性能指标

系统主要指标见表 1。计算平台为 ALINX Z19，PL 统一工作于 ps0/pl_clk1 目标 100 MHz，四功能集成工程已按该频点完成完整实现；视频最大支持 4096×4096，计数 32 位、一阶矩 64 位；音频为 16 kHz 单声道 16 位直流抑制使用 40 位 Q24.15 状态，FIR 为 63 抽头、Q1.15 系数与 48 位累加。

表 1 主要性能指标

项目	指标	条件与说明
计算平台	ALINX Z19	XCZU19EG-FFVC1760-2-I； 4×Cortex-A53 + 可编程逻辑
逻辑时钟	100 MHz	ps0/pl_clk1，同域 proc_sys_reset 复位

项目	指标	条件与说明
视频通路	1280×720 采集 → 640×360 处理	UVC/V4L2 解码，PIO 逐像素进入 PL
板端发布率	7.26 fps（balanced）	原 hd 配置为 1.924 fps
采集至发布	约 187 ms（balanced）	原 hd 配置为 567.99 ms
视频统计	最大 4096×4096	count 32 位、一阶矩 64 位，末像素闭合
音频前端	16 kHz / mono / S16_LE	DC 40 位 Q24.15；FIR 63 tap、48 位累加
任务时限	唤醒有效期 20 s；识别上限 20 s	目标需连续 3 帧稳定出现
任务接口	JSON + Bearer；2 MiB；3 s	最多 8 个处理线程，JPEG 512 KiB
资源与时序	7,292 LUT / 8,882 FF / 14 DSP / 0 BRAM	四功能集成完整实现，100 MHz 下 WNS +0.632 ns
片上功耗	4.167 W（动态 2.955 / 静态 1.212）	Vivado vectorless 估算，非板上测量

1.6 主要创新点

① 候选门控：以检测框面积、严格新帧、首帧锚点抖动与一次消费代替逐帧触发，误抓与重复执行减少，每次动作都能回溯到具体帧。

② 在途信用：未出结果的输入也计入 FIFO 压力，参数四态快照、末像素闭合统计；背压时不丢像素、不串帧，换分辨率按公式重算容量即可。

③ 资源反推的定点设计：位宽按最坏边界推导，63 抽头 FIR 由 63 个乘

法器改为 1 个时分复用；整机只占 1.40% LUT、0.71% DSP，为 DMA 与更多算子留出更多余量。

④ 单清单三端同源：registers.json 生成 RTL/C/Python 投影并在打开时核对 ID 与版本；改一处三端同生效，接口错配能及时发现。

⑤ 语音边界：boundary_id 串联采音、管道、滤波与识别，控制独占设备展示只读；半句口令不会补成下一轮唤醒，界面卡顿不影响控制。

1.7 设计流程

设计从“一次任务能否被完整观察”出发：先确定固定工位与单轮任务边界，再从配套工程中提取真实的模型调用与机械动作接口；随后按计算性质划分 PS 与 PL，依次完成视频、音频、候选与任务四个模块；每个模块都要求仿真或现场记录先于文字结论；最后以接口审计发现的矛盾驱动修订，并把可复用的经验沉淀为技能包。图 2 给出阶段划分与各阶段产出。



图 2　设计流程：需求、划分、实现、验证、修订与交付六个阶段

第二部分　系统组成及功能说明

2.1　整体介绍

图 3 给出系统整体框图。系统自上而下分为应用层、接口层、PS 软件层、PL 逻辑层，并由外设与执行机构提供图像、声音与动作。PL 承担定点运算、区域统计与候选门控，PS 承担采集、语义、坐标与口令，接口层把两侧的状态与数据组织成一次任务，应用层负责编排、播报与只读呈现。

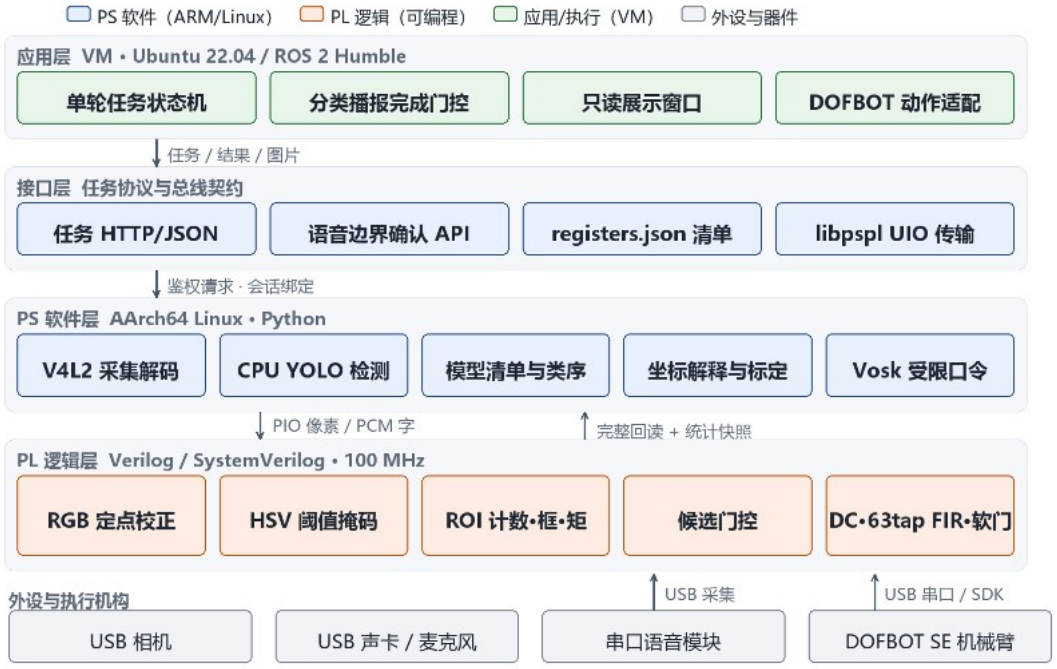


图 3　系统整体框图：四层分工与外设关系

层间有两条主通路。数据面由 PS 把像素与 PCM 写入 PIO FIFO，PL 处理后完整回读；控制面由 AXI-Lite 寄存器承载配置、状态、计数与 ACK，寄

存器定义来自同一份 JSON 清单。应用层通过带鉴权的 HTTP/JSON 与 PS 服务交互，机械动作仍由上位机中既有的 ROS/SDK 完成。

这一划分使“哪里算、哪里判、哪里动”各有明确边界：PL 的结论是统计量与候选许可，PS 的结论是类别、坐标与任务状态，上位机的结论是动作指令与界面显示。三者的输出格式、时序条件与失效方式在后续各节分别说明。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

整机由三个功能簇组成（图 4）：感知与输入包括 UVC 相机、USB 声卡与麦克风、串口语音模块；计算单元由 Z19 与上位机组成，两者通过千兆以太网交换任务与结果；执行与人机交互包括 DOFBOT SE 六轴机械臂、其控制器、独立供电与显示/扬声器。



图 4　硬件组成与连接：感知、计算、执行三个功能簇

表 2　硬件设备与接口

设备	接口	用途	核对要点
ALINX Z19	USB / 以太网 / SD	PS Linux 与 PL 通路	位流、设备树与 UIO 映射
USB 相机	USB 2.0（UVC）	现场画面与识别图像	by-id 固定与分辨率
USB 声卡与麦克风	USB Audio	语音输入与分类播报	ALSA 设备与采样格式
串口语音模块	UART / USB 串口	唤醒与启动命令	波特率与编号字段
DOFBOT SE 机械臂	USB 串口 1a86:7523	抓取与分类投放	独立供电、单一实例
上位机（VM）	以太网 / USB 透传	任务编排与展示	ROS 版本与依赖来源

时钟与复位：视频、音频与门控外设统一使用 ps0/pl_clk1，目标 100 MHz，同域 proc_sys_reset 释放；16 kHz 音频以样本握手表达，不引入额外时钟域。地址规划为 0xA100_0000 视频、0xA101_0000 音频、0xA102_0000 门控，各 4 KiB，与 profile.json、Tcl 脚本和设备树保持一致。

2.2.2 机械设计介绍

机械部分沿用 DOFBOT SE 的既有结构与动作能力，团队的设计工作集中在工位布局与约束：相机固定于工位上方，视场覆盖固定拾取位与分类投放区拾取位以工位垫上的圆角方框标定，样本放入后须移开手；四个投放区分别对应可回收物、有害垃圾、湿垃圾与干垃圾。

表 3　固定工位与机械参数

项目	取值 / 做法	说明
拾取位	工位垫上圆角方框标出的固定区域	单件放置、居中，便于复位
相机安装	悬臂支架，镜头俯视样本区	视场覆盖样本区与四个标识区
工位垫	印有四类标识（可回收物·有害垃圾·湿垃圾·干垃圾）	整机可搬移，铺开即回到同一布局
标定板	400×300 mm，大格 100 mm、小格 20 mm	X 向右、Y 向下，左上角为原点
样本	约 3 cm 立方体，印有物品名称与类别	便于讲解、替换与重复演示
投放区	垫面标识 + 前后四个收集盒	机械臂按类别投放，换桶即换去向
动作方式	固定工位类别动作	沿用配套 SDK 轨迹，不依赖任意位置规划
安全措施	独立 12 V 供电，可物理断电	动作期间人员与杂物离开动作区域



图 5　工位垫与四类标识：可回收物、有害垃圾、湿垃圾、干垃圾与固定样本区

图 5 是工位垫。垫面印有四类标识与系统名称，中央圆角方框为固定样本区，四类标识沿垫边排布，与界面中的四个投放区一一对应；现场演示时在垫的前后放置橙、蓝、棕、绿四个收集盒，机械臂按类别投放。讲解只需把样本放回方框、指向对应标识，讲解与复位都落在同一张纸面上。

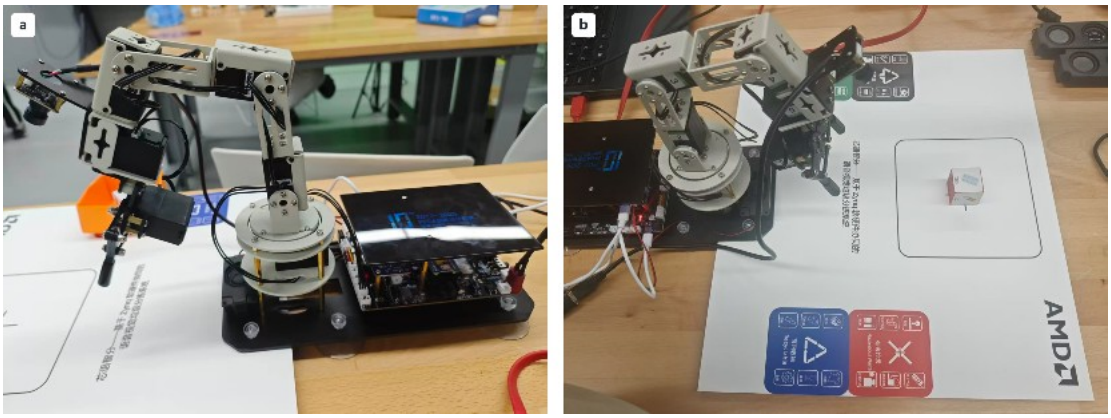


图 6　机械臂与末端执行器：(a) 夹爪、吸盘与腕部相机；(b) 机械臂、控制盒与工位垫

图 6 给出末端与机械臂。末端执行器同时装有夹爪与真空吸盘：夹爪夹持立方体样本，吸盘用于平整表面的取放；腕部另装小型相机模块，在抓取前提供近距离辅看。机械臂底座与带小屏的控制盒固定在工位垫左侧，与垫面保持刚性相对位置，因此换点铺开后不必重新标定。

2.2.3 电路各模块介绍

电路按板卡供电、外设接口、调试与存储三部分组织。Z19 由 12 V 直流输入，板上产生各轨电压；机械臂使用独立电源，避免动作电流影响板卡与相机。相机与声卡经 USB 接入，串口语音模块经 UART 或 USB 转串口接入；SD 卡承载镜像与启动，JTAG 与调试串口用于下载和观察。

表 4 电气与总线接口约定

接口	电平 / 协议	方向	工程说明
12 V 输入	直流	输入	板卡供电，与机械臂电源独立
USB 2.0 / 3.0	UVC / UAC	双向	相机图像与声卡音频
UART	3.3 V TTL	双向	命令编号，兼作调试串口
千兆以太网	RGMII	双向	与上位机交换任务与结果
AXI4-Lite	32 位 / 4 KiB 映射	双向	PS 访问三个外设
PIO FIFO	32 位数据字	双向	像素与 PCM 逐字收发

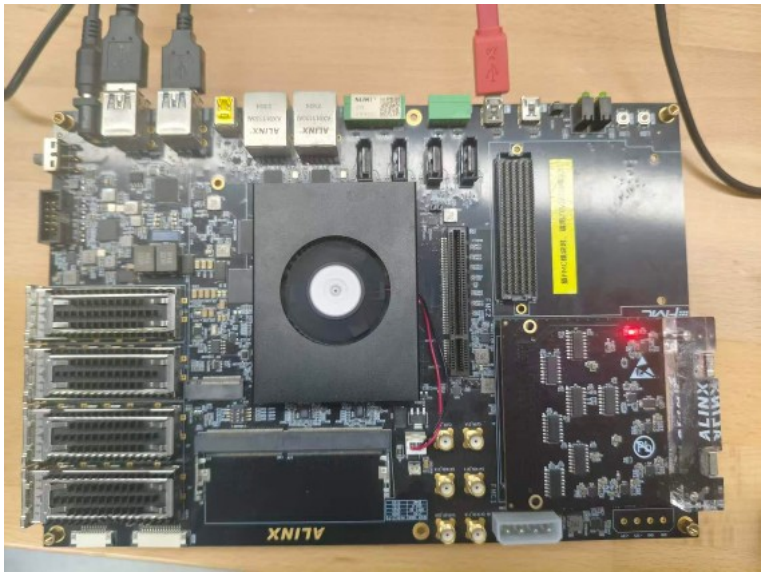


图 7　Z19 板卡俯视：核心板、FMC 扩展卡与板边接口

图 7 为板卡俯视。核心板居中并加装散热器与风扇，右侧接一块 FMC 扩展卡，左侧为 PCIe 与高速连接器，板边依次是 USB 3.0、千兆以太网、HDMI、SD 卡槽与直流电源输入。红色电源线从桌面绕行，避免线缆横跨样本区；调试串口与 JTAG 引出后留在板卡侧边，便于观察启动日志和重新下载。

2.3　软件系统介绍

2.3.1　软件整体介绍

软件按四层组织（图 8）：应用层位于上位机，负责一次任务的编排、分类播报与只读展示；服务层位于 PS，负责任务 API、候选策略、坐标解释与口令事件；传输层提供有序 32 位 MMIO、FIFO 填排与错误传播；驱动与硬件

层包括 V4L2、ALSA、UIO 与 PL 算子。



图 8　软件分层与源码入口

表 5　软件模块与源码入口

层次	模块	职责	源码入口
应用层	单轮任务 / 播报 / 展示	状态机、播报门控、只读界面	robot/dofbot
服务层	识别、定位、候选、口令	模型调用、坐标、策略与事件	host/app/perception
传输层	libpspl 与 Python 绑定	UIO 映射、PIO 编排、超时	host/libpspl
驱动层	V4L2 / ALSA / UIO	采集、采音、寄存器访问	host/camera_linux、host/app/voice
硬件层	PL 算子与门控	定点运算、统计、候选许可	fpga/perception_v2、fpga/audio_frontend

2.3.2　软件各模块介绍

本节按数据流顺序说明各模块的设计：视频从像素到统计，识别从图像到

类别，定位从统计到坐标，门控从候选到许可，语音从 PCM 到口令，最后是任务协议与状态机。每个模块给出关键算式、位宽、握手条件与失败处理。

2.3.2.1 视频预处理与区域统计

PS 将 UVC 相机帧解码为 RGB888，并线性缩放至 profile 尺寸后送入 PL。每个 PIO 字低 24 位为 0xRRGGBB，bit24 为帧首 SOF，bit25 为行末 EOL，其余位为 0；输出保留这些位，并在 bit26 返回颜色掩码。

通道校正在 PL 逐像素完成，gain 为 Q8.8 无符号数（0～1024，默认 256），bias 为 −255～255 的有符号整数（默认 0）：

```
out = clamp( ((pixel × gain + 128) >> 8) + bias , 0 , 255 )
```

运算顺序为先对非负乘积舍入，再扩展符号并加偏置，最后饱和到 0～255，因此负偏置与中间位宽都不会造成截断。HSV 算术复用固定来源的 laperex MIT 实现，团队补充了任意合法阈值、跨零色相（Hmin > Hmax）、掩码生成与流控；色相范围 0～179，饱和度与亮度 0～255。

ROI 只筛选统计，不裁剪像素流。统计在像素流经时同步累加：count、包围框 xmin/ymin/xmax/ymax，以及一阶矩 sum_x、sum_y。最大 4096×4096 时

count ≤ 2^{24}，一阶矩 < 2^{36}，故采用 32 位计数与 64 位矩；count 为 0 时包围框清零，PS 返回空质心。

表 6　视频像素字与统计字段

字段	位置 / 位宽	含义
像素字	bit23:0	0xRRGGBB 颜色值
SOF / EOL	bit24 / bit25	帧首 / 行末标记
mask	bit26（输出）	当前像素是否命中阈值
count	32 位	ROI 内命中像素数
bbox	4×16 位	xmin / ymin / xmax / ymax（闭区间）
sum_x, sum_y	2×64 位	命中像素的坐标一阶矩
config_version	32 位	当前生效配置编号，冻结至下一帧



图 9　视频数据通路与整帧事务：像素入 PL、统计回读、快照与 ACK

流控的关键是“在途信用”。普通弹性级只在 valid 与 ready 同时为真时

消费数据；复用的 HSV 算术核自由运行，标签必须按真实算术延迟逐周期推进，因此不能靠一次停顿把算术与标签一起冻结。设 FIFO 容量为 CAP，已占用与已预留槽数为 O，本周期接受的输入为 I，实际消费的输出为 D，则接受条件为：

$$O + I - D + 1 \le CAP$$

其中“已预留”包含已进入算术流水、尚未产生结果的输入。该方法把“结果尚未到达”也纳入容量计算，使下游停顿不会造成溢出或标签错位。

配置采用 staging、pending、active 三层：全部参数校验提交后进入 pending，接受帧首时切换为 active，并保持到完整帧回读和统计 ACK。首像素立即使用所属帧参数；快照在末像素写入后才发布，实测输出消费数达到整帧数量后才算完整；行长或 EOL 异常、中途复位及部分帧不会发布为完整有效结果。

源码入口：fpga/perception_v2/rtl/vg_rgb_affine_axis.sv、vg_hsv_mask_axis.sv、vg_roi_moments_axis.sv、vg_video_axi.sv；host/app/perception/coordinates.py

2.3.2.2 垃圾识别分类

识别在 PS 的 CPU 上运行真实 Ultralytics YOLO 检测后端。适配器显式

调用 task='detect'，沿用配套程序的 640×480 BGR 外层约定与 0.65 置信度；letterbox、归一化与 NMS 由匹配版本的后端完成，外层只解析 post-NMS 的 [xmin, ymin, xmax, ymax, confidence, class_id] 并撤销外层缩放，不重复执行 NMS。

加载前先核对模型清单：权重 SHA-256、有序类名、源与目标环境、框架文件哈希、内层尺寸与步长、NMS 参数与预处理契约。加载时再次比对 model.names 与实际 stride，不一致即报错；调用成功但没有检测框，与后端异常是两种不同状态，分别返回。

表 7　模型清单契约字段

契约字段	作用	核对方式
weights_sha256	权重指纹	加载前比对文件哈希
names / names_sha256	有序类名	与 model.names 逐一比对
framework_files	加载器、预测器、NMS 等来源	逐文件哈希，运行时同一清单
source/target_environment	源环境与目标环境	x86 VM 与 AArch64 分别登记
image_size / stride	内层输入尺寸与步长	尺寸按 stride 对齐并核对
iou / max_detections	NMS 参数	取自真实框架，不用默认猜测
preprocess	色彩、布局、归一化与 letterbox	与后端实现一致

类别到动作的映射由业务表给出：16 个标签分为四组，分别对应可回收、

其他、厨余与有害四类投放动作；另有 8 个标签被明确排除，不参与任务。业务表的排列不是模型的 class ID 顺序，修改中文显示名也不会改变模型的识别能力——这两件事必须分开处理。

源码入口：host/app/perception/detector.py、preprocess.py、postprocess.py、model_manifest.py、class_mapping.py

2.3.2.3 视觉定位与候选策略

定位模块同时保留三种坐标。检测框中心是语义框两边界的平均位置；匹配像素质心由 PL 的一阶矩除以命中像素数得到，count 为 0 时为空；table_xy_mm 是相机平面标定后的桌面坐标，必须绑定 camera_id、transform_id、image_size 与 calibration_id。像素逆映射按线性缩放给出，例如 1280×720 到 640×360 的变换为：

$$x_{cap} = 2 \cdot x_{proc} + 0.5 \ , \quad y_{cap} = 2 \cdot y_{proc} + 0.5$$

候选策略由双端显式配置并核对，默认 semantic_bbox 面向垃圾语义任务当同帧存在唯一真实检测目标时，把该框覆盖的整数像素区域与面积 bbox_area_px = (xmax − xmin + 1)(ymax − ymin + 1) 交给 PL 门控，颜色掩码与颜色质心独立报告，不参与否决。color_association 只用于明确要求颜色的工位：

除唯一语义目标外，还要求存在唯一且完整落在语义框内的 PL 连通区域，计数改用真实匹配像素数 pl_mask_pixels。

表 8　两种候选策略对比

策略	候选条件	计数依据	颜色的作用
semantic_bbox（默认）	同帧唯一语义检测框，闭区间整数框	bbox_area_px（PS 计算）	独立报告，不否决候选
color_association	唯一语义目标 + 唯一且完全包含于框内的 PL 连通区	pl_mask_pixels（PL 统计）	指定颜色为必要条件

平面映射采用单应变换，输入输出都绑定图像尺寸与标定编号；标定不匹配时保留像素坐标并记录原因。桌面平面坐标不解释为机械臂基座姿态，也不直接生成任意位置的抓取轨迹——固定工位的动作仍由类别决定。

源码入口：`host/app/perception/service.py`、`coordinates.py`、`guard_adapter.py`；`robot/dofbot/pspl_client.py`

2.3.2.4　候选门控

检测回答“看到了什么”，任务还需要回答“这个结果现在能不能用”。候选门控把这组条件实现为可综合的 RTL：工位 ROI、计数区间、严格新帧、固定锚点抖动、周期超时、busy 撤销与每轮一次的候选许可。图 10 给出 ARM→判定→ready→CONSUME 的完整流程。

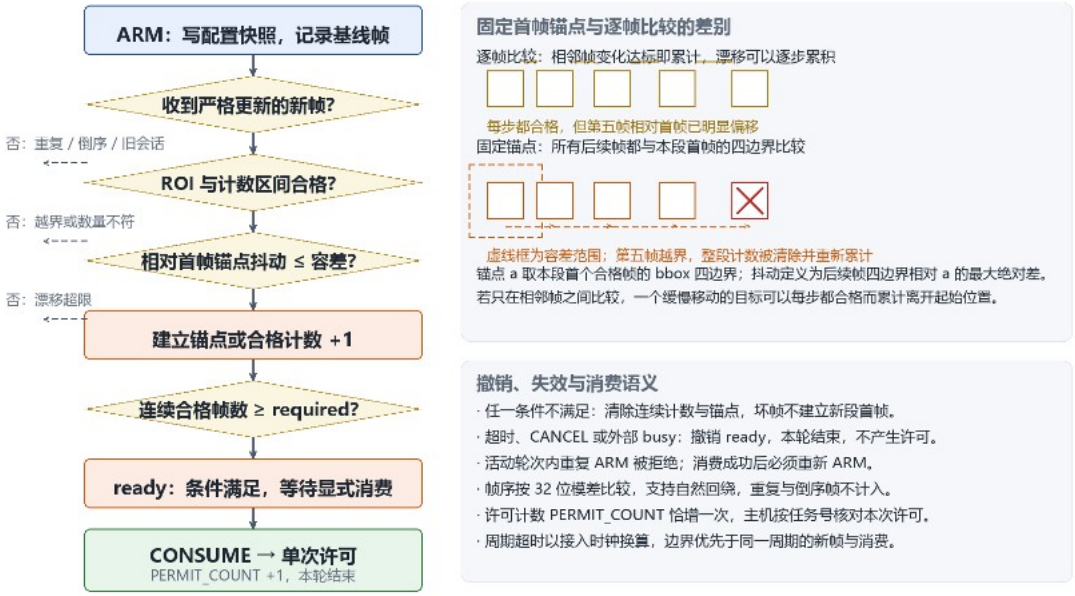


图 10　候选门控流程与固定锚点判据

ARM 时锁存配置快照与基线帧，活动轮次内再次 ARM 会被拒绝。新帧必须比基线或上一帧严格更新：帧序号按 32 位模差比较，$0 <$ 新−旧 $< 2^{31}$ 才算新帧，既支持自然回绕，也拒绝重复与倒序。抖动判据取当前候选段首个合格帧的四边界为锚点 a，后续帧的四个边界相对 a 的最大绝对差不超过容差才计入连续计数。

固定锚点的意义在于约束整段漂移。若只比较相邻两帧，一个缓慢移动的目标可以每一步都合格，却累计离开起始位置；以首帧为锚点后，漂移不再被逐帧稀释。任一条件不满足即清除连续计数与锚点，该坏帧不建立新段的第一个锚点，从而避免用坏帧重置基准。

表 9　候选门控规则

判据	条件	不满足时
空间	bbox 完整落在 ROI 闭区间内	清除连续计数与锚点
数量	匹配像素数或检测框面积在 [min, max] 内	清除连续计数与锚点
时间	帧序号严格更新（32 位模差）	不计数、不延长有效期
稳定	连续 required 个合格新帧	继续等待下一新帧
漂移	四边界相对首帧锚点 ≤ 容差	清除本段并重新累计
超时	周期计数超限（接入时钟换算）	撤销 ready，本轮结束
消费	显式 CONSUME 一次，PERMIT_COUNT 恰增一次	重复消费失败，须重新 ARM

寄存器侧为 32 位 AXI4-Lite：AW 与 W 独立接收、B/R 响应在有背压时保持、读副作用只在 AR 被接受时执行一次；非法地址、部分写与读写溢出均返回 SLVERR 且不产生数据副作用。许可输出为单周期脉冲，同时在 PERMIT_COUNT 与 PERMIT_JOB 留存，主机核对计数恰增一次与任务号后，才认定这次许可属于本轮任务。

源码入口：`fpga/sort_guard/rtl/vg_sort_guard_core.sv`、`vg_sort_guard_axi.sv`；`host/app/perception/guard_adapter.py`

2.3.2.5　语音前端与受限口令

语音有两路输入，语义统一。串口语音模块直接给出命令编号，作为事件

源接入；USB PCM 模式读取 16 kHz、单声道、S16_LE 采样，经 PL 的直流抑制、63 抽头 FIR 与可选软噪声门后，由 PS 的 Vosk 解析为受限中文口令。两路互斥选择，命令编号不经过 FIR。

直流抑制使用 40 位 Q24.15 状态与扩展反馈乘积，差分输入扩为 17 位：

$$d[n] = x[n] - x[n-1] + (32604 / 32768) \cdot d[n-1]$$

FIR 为 63 抽头、Q1.15 系数、48 位有符号累加，最坏累加幅值不超过 $63 \times 32768 \times 32768 < 2^{36}$，48 位余量充足。为节省乘法结构，63 次乘加由一个乘法器按 tap 顺序时分复用：目标时钟 100 MHz、采样率 16 kHz 时，每个样本约有 6,250 个时钟周期，63 次乘加在预算内留有余量。系数由设计函数生成——63 点 Blackman 窗带通 sinc，初始通带 150～6500 Hz，在 1 kHz 归一化后量化为 Q1.15，并以 SHA-256 记录配置身份。

软噪声门默认关闭，开启时使用双阈值迟滞：17 位幅度包络瞬时上升、逐样本下降 max(envelope >> 6, 1)，达到开阈值开门、降至关阈值关门，中间区间保持状态；增益为 UQ1.15 编码 0～32768，可精确表示 0 与 1，开门按 attack、关门按 release 步长逐样本改变。所有定点舍入统一为最近、半值远离

零，PCM 输出与状态均做饱和。

表 10　音频位宽与参数

参数	取值	说明
采样格式	16 kHz / mono / S16_LE	USB 声卡经 ALSA 采集
直流抑制	40 位 Q24.15 状态，反馈乘积 56 位	差分输入 17 位
FIR	63 tap，Q1.15 系数，48 位累加	单乘法器时分复用
通带	150～6500 Hz，Blackman 窗	1 kHz 归一化后量化
软门	双阈值 + 包络 + attack/release	UQ1.15 增益，默认关闭
舍入	最近、半值远离零	PCM 输出与状态饱和
剪裁计数	CLIP_COUNT，uint32 饱和	任一通道饱和时累加

口令与任务语义仍为“你好小亚”唤醒、等待回应“在的”、再说“开始垃圾分拣”启动；“开始分拣”作为兼容表达保留，“停止 / 取消任务”用于受限取消。部分识别或低置信度不产生事件。滤波状态跨普通 PCM 块保持，普通块边界不会清空历史。

语音的关键设计在任务边界。任务开始与结束时，上位机通过鉴权的 /voice/boundary 请求生产端重建：先中断当前采集读取，丢弃在途 PCM 块，关闭旧进程与管道，重新打开 PL 滤波会话并清除滤波历史，Reset 识别器，再启动新采集后才确认递增的 boundary_id。每个事件携带其实际所属的边界，旧

事件不会被改写成新边界。忙碌期间录入的半句口令、管道缓存与旧 ASR 状态因此不能在任务结束后补成新的唤醒。



图 11　音频前端与语音生产端边界

源码入口：`fpga/audio_frontend/rtl`；`host/app/voice/coefficients.py`、`pl_frontend.py`、`usb_pcm_pl.py`

2.3.2.6　传输、寄存器与任务接口

寄存器偏移与位字段只在 registers.json 维护：RTL 使用对应的 registers.svh，C 使用 registers.h，Python 直接读取该 JSON，三者由同一生成脚本投影。公共区域给出设备 ID、协议版本、能力位、控制与状态、错误（W1C）、输入/输出计数、数据窗口与配置版本；非法地址、非对齐访问与部分 WSTRB 一律 SLVERR。

PS 侧的 C 客户端 libpspl 通过显式 UIO 节点执行有序 32 位 MMIO：

打开时核对 ID、协议版本与 map 尺寸，运行中交替填输入、排输出，核对读写数量、帧序号、SOF/EOL 与快照有效位，超时或数量不符即终止会话，不做部分结果拼接，也不在失败时静默回退到软件处理。

表 11　任务服务接口

接口	方法	用途	关键约束
/task	POST	发起一轮任务	task_id；20 s 任务上限；候选策略双端一致
/result	GET	读取同帧识别、定位与许可	会话、帧号、相机、变换与配置绑定
/cancel	POST	撤销当前任务	撤销后不产生许可
/voice	GET	读取受限口令事件	事件带音频源、块序与边界号
/voice/boundary	POST	确认语音生产端已重建	8 s 截止，递增 boundary_id
/preview	GET	原始采集只读旁路	标记 pl_processed=false，不参与许可
/stop	POST	服务退出	不强杀在用的模型线程或 MMIO

所有请求使用 JSON 与 Bearer token，单报文上限 2 MiB，JPEG 上限 512 KiB，连接读写超时 3 s，最多 8 个处理线程；跨机时只传递生产端数据年龄，接收端叠加自身请求与解码耗时判断时效，不直接相减两台机器的单调时钟。

2.3.2.7 单轮任务状态机与只读展示

任务状态依次为等待唤醒、等待启动、准备观察、稳定识别、分类播报、抓取投放，结束后回到等待唤醒（图 12）。每轮开始消耗一次唤醒状态，代际编号与输入清理阻止忙碌期间或上一轮的命令触发下一轮；唤醒有效期为 20 s。识别要求目标在时间与位置约束下连续稳定出现 3 帧，上限 20 s；分类播报阻塞至播放完成，失败则不进入抓取。

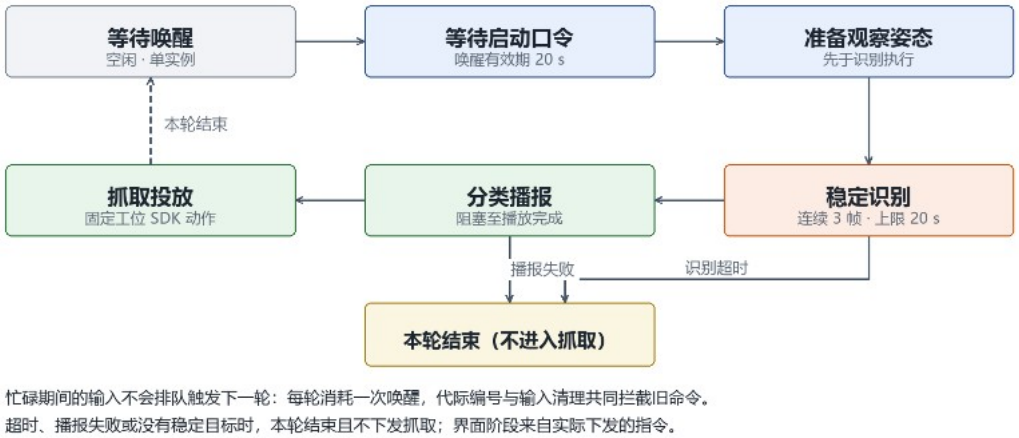


图 12　单轮任务状态机与失败分支

展示与控制解耦：控制进程唯一拥有机械臂、语音串口与相机；显示线程旁路复制最新帧，以约 8 Hz 发布 JPEG 与元数据，不消费控制取帧游标。窗口只读取状态与预览，不直接打开硬件，也不提供点击触发运动的入口。预览

携带进程、会话、时间戳与帧序号，窗口核对运行进程与数据新鲜度，旧会话编码失败或过期画面不显示为 LIVE。动作阶段通过成功下发的 SDK 调用观察可显示抓取、转运、释放与返回；F11 切换全屏，重复启动只唤起已有窗口。

源码入口：robot/dofbot/standalone_voice.py、preview_publisher.py、dashboard.py、launch_controller.py

2.4 关键算法与创新设计

本节的四项设计分别回答“判得准、算得稳、算得省、管得住”，也是本作品相对“通用检测 + 机械臂”方案的主要区别。

2.4.1 从“识别”到“允许动”：候选门控的可核对判据

检测是逐帧的、概率的，执行是一次性的、物理的，二者之间需要一个可核对的判据。作品把判据拆成四类：唯一语义目标与面积区间（空间与数量）32 位模差严格新帧（时间）、相对首帧锚点的四边界抖动（稳定）、显式 CONSUME 的一次许可（消费）。四类条件全部落在寄存器与日志里，任何一次执行都能回溯到“哪一帧、多大面积、多少抖动”。

为什么用首帧锚点而不是相邻帧比较。设首帧四边界为 a，第 k 帧为 b_k，抖动定义为 d_k = max|b_k − a|。若改成相邻帧比较，每步允许 ε，n 步之

后累计位移可达 nε——一个缓慢平移的目标可以“每步合格”地离开工位；以 a 为锚点后，整段漂移被同一个界约束，越界即清空本段计数，且坏帧不用于重置锚点。

2.4.2 在途信用与整帧事务：让定点流水在背压下不串帧

复用的 HSV 算术核没有 ready/clock-enable，标签必须按真实算术延迟逐周期推进，因此不能靠一次停顿把算术与标签一起冻结。作品按“已占用 + 已预留”核算 FIFO 压力：$O + I - D + 1 \le CAP$，其中已预留包含已进入流水、尚未产生结果的输入。代价是容量要比“只看已到达结果”更大，收益是下游停顿时既不溢出也不串帧。

帧一级用四态快照把参数与数据绑在一起：staging → commit → pending → active，首像素使用所属帧参数，统计在末像素闭合后冻结，主机读取并 ACK 才接纳下一帧；行长或 EOL 异常的帧不会发布为完整结果。

2.4.3 面向资源的定点设计：位宽由最坏边界反推

位宽不是试出来的。最大 4096×4096 时 $count \le 2^{24}$、一阶矩 $< 2^{36}$，故计数用 32 位、矩用 64 位；直流抑制的反馈状态保留 15 位小数，40 位 Q24.15 覆

盖最坏反馈乘积；63 抽头 FIR 的最坏累加不超过 $63\times32768\times32768 < 2^{36}$，48 位累加器留有余量。63 次乘加由一个乘法器按 tap 顺序时分复用：100 MHz、16 kHz 下每样本约 6,250 个时钟周期，预算充裕，换来 DSP 与布线的节省。

完整实现的结果印证了这套推导：7,292 LUT（1.40%）、8,882 FF、14 DSP（0.71%）、0 BRAM，100 MHz 下 WNS +0.632 ns。

2.4.4 单清单寄存器与生产端语音边界：让系统可被复核

寄存器偏移、位字段与设备 ID 只在 registers.json 维护，RTL、C 与 Python 由同一脚本投影；打开设备先核对 ID、协议版本与 map 尺寸，非法地址与部分写返回 SLVERR，读副作用只在 AR 被接受时执行一次。三端不会各自解释同一位，接口错误也会在第一时间暴露。

语音的问题在缓存而不在识别：任务忙碌时录入的半句口令可能滞留在管道、滤波状态或 ASR 内部，任务结束后被“补”成新的唤醒。作品以 boundary_id 把采音进程、管道、滤波会话与识别器串成一代，任务开始与结束都重建一代，旧事件不能改写成新边界。

表 12　创新点、常规做法与证据对照

创新点	常规做法	本作品的做法	可核查证据
候选判据	检测框直接触发动作	面积 + 严格新帧 + 锚点抖动 + 一次消费	661 次断言、94 项离线回放
流水控制	只看已到达结果的 FIFO	在途信用 + 四态快照 + 末像素闭合	RTL 仿真、输入/输出计数
定点位宽	试凑或统一位宽	由最坏边界反推，单乘法器时分复用	7,292 LUT / 14 DSP / WNS +0.632 ns
接口与语音	各处自定义；接收端清缓存	单清单三端投影 + 生产端 boundary_id	寄存器 ID/版本校验、边界回执

这些设计的意义不止于指标。分拣现场的重复搬运、粉尘与尖锐物是长期存在的职业健康风险，把识别、候选与拾取交给工位，人可以只保留抽检、换桶与讲解；分类结果第一次被逐条记录，回收纯度、湿垃圾分出量与有害垃圾拦截量也就有了可以统计的口径。

2.5　软硬件划分依据与接口设计

划分依据不是“能放进硬件就放”，而是三条准则：计算的规律性、依赖的可获得性、失败的可观察性。

（1）规律性。逐像素的通道校正、HSV 比较、计数与矩累加，逐样本的直流抑制、63 抽头 FIR 与软门，都是定序、定点、无分支的密集运算，适合 RTL 流水或时分复用；放进 PL 之后，PS 不必为每个像素或样本做一次系统

调用。

（2）依赖。模型框架、词典、USB/V4L2/ALSA 驱动与网络协议栈依赖 Linux 生态和持续更新的库，留在 PS 才能随版本演进；把它们固化进 PL 既不可行，也不必要。

（3）可观察性。机械动作、播报与界面依赖厂商 SDK 与 ROS 环境，放在上位机可以沿用已验证轨迹；一旦下沉到 PL，失败原因就难以定位。三条准则的权衡结果是：PL 只回答“这一帧的统计与许可”，PS 回答“这是什么、在哪、任务到哪一步”，上位机回答“做什么动作、怎么呈现”。

接口按五个面设计，每一面都有明确的失效行为。数据面为 32 位 PIO 字：低 24 位是 RGB，bit24 为 SOF、bit25 为 EOL，输出 bit26 返回掩码；PCM 使用低 16 位有符号样本，高 16 位必须为零。控制面为 AXI-Lite：32 位、全 WSTRB、AW 与 W 独立接收，偏移与位字段由 registers.json 单一维护，非法地址、非对齐与部分写返回 SLVERR 且不产生副作用。

时间面按帧序与年龄处理：帧序用 32 位模差比较，数据年龄由生产端给出、接收端叠加自身耗时，不跨机相减两台机器的单调时钟。身份面把

camera、session、frame 与 config 绑在结果上，模型输入与展示高亮分别记录任务面为 HTTP/JSON + Bearer：单报文 2 MiB、连接超时 3 s、最多 8 个处理线程，语音边界用 /voice/boundary 与 boundary_id 确认。

失败处理一律显式：超时或计数不符即终止会话并 reset，不把部分结果当完整帧，也不在失败时静默回退到软件实现。这条约定让“哪里算错了”可以定位到具体模块，而不是变成一次无法复现的偶发问题。

第三部分　完成情况及性能参数

3.1　整体介绍

整机由三条工程线组成并分别留有记录：固定工位演示（现场运行）、Z19 独立感知与网格坐标（现场运行）、PS/PL 四功能集成工程（完整实现与 bit/XSA 交付）。三条线共用同一套任务语义与展示方式，配置、镜像与产物各自独立，互不覆盖。

表 13　交付内容与完成情况

交付对象	主要内容	当前状态
固定工位演示	冻结安装源 15 个文件、安装与启动脚本、只读界面、任务记录	现场运行，78 项离线检查通过
Z19 独立感知	颜色/网格/坐标播报记录、仿真与综合报告	现场运行，发布率与资源有原始报告
四功能集成工程	视频/音频/门控 RTL、libpspl、Tcl、PS 服务、bit 与 XSA	综合、实现与 bitstream 完成；WNS +0.632 ns
验证与文档	证据记录、技能包、复现步骤与配置示例	随仓库交付

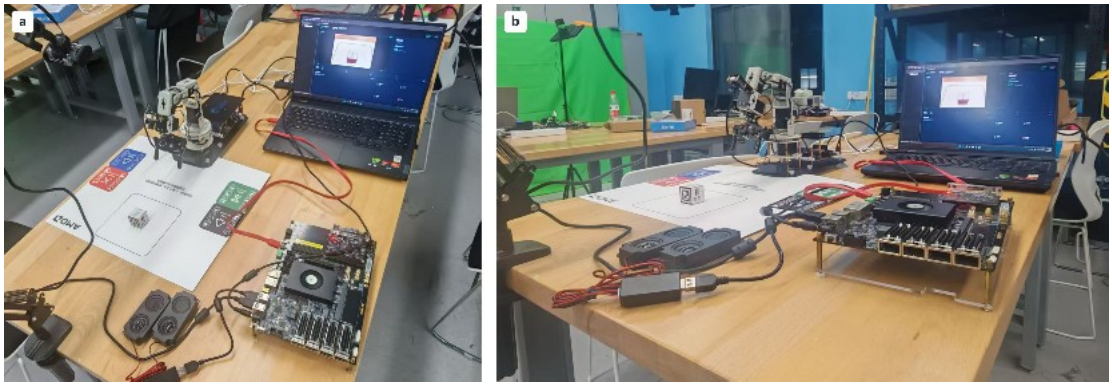


图 13　整机实物：(a) 俯视；(b) 斜 45°

图 13 是交付形态的整机。(a) 俯视可见工位垫、机械臂与控制盒、悬臂相机、双扬声器、Z19 板卡与上位机；(b) 斜视可见整机只占一张工位桌，两侧留出讲解与通行空间。断电收线后可整体搬移，到下一个教室、展台或垃圾站临时分拣位重新铺开——这正是“工位级单元、可复制节点”的物理含义。

3.2 工程成果

3.2.1 机械成果

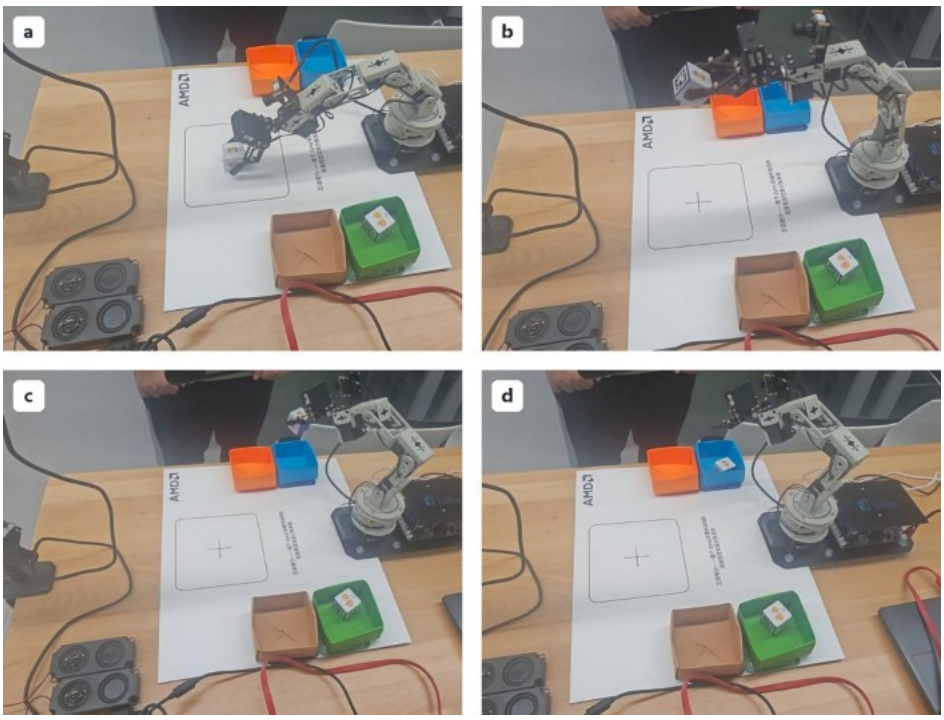


图 14 一次任务的动作序列：(a) 定位并抓取样本；(b) 抬升离开样本区；(c) 转运至对应收集盒；(d) 投放完成并返回

图 14 记录了一轮完整动作：机械臂在样本区上方闭合夹爪，抬升后转向收集盒，在盒口上方释放并返回等待位。四个画面取自同一次任务，顺序与界面中的“抓取投放”阶段一致。

固定工位在桌面上以工位垫确定拾取位，四个投放区沿垫边布置，机械臂底座与悬臂相机支架刚性固定。工位的特点是“复位即可重复”：每次任务只

需把样本放回方框中心，不必重新标定相机或调整轨迹，因此同一套参数可以连续接待多位体验者。

机械结构的动作由类别决定，四组业务类别对应四条既有轨迹。夹爪在拾取位闭合、提升、转运到对应投放区后释放并返回等待位，整个过程在窗口中以“抓取投放”阶段呈现。

3.2.2 电路成果

电路成果包括 Z19 与上位机的连接、外设接入与供电分离。板卡与机械臂使用两路独立电源，动作电流不会通过板卡回流；相机、声卡与语音模块的 USB 连接在上电后透传至虚拟机，串口设备以别名固定，避免 ttyUSB 编号变化带来的设备错配。

板端侧固定了三块 4 KiB 的 AXI4-Lite 映射与对应的设备树节点，位流、XSA 与 UIO 名称一一对应；调试串口与 JTAG 保留在外部，便于观察启动日志与下载镜像。

3.2.3 软件成果

软件成果的最终形态是固定工位分拣界面（图 15）。界面左侧为现场视野

与 LIVE 状态，右侧为识别结果与投放类别，中部是任务流程的五段进度，下方依次是语音与任务记录、四个投放区状态与最近一次识别画面。记录按时间排列，可以回看某一轮从唤醒到完成的全部事件。

界面之外，工程交付还包括：四功能 PS/PL 源码（视频、音频、门控与系统工程）及其 bit 与 XSA、libpspl 的 C 与 Python 传输层、模型清单与入册工具、任务服务与 VM 入口、四份可复用技能，以及配置示例与复现步骤。

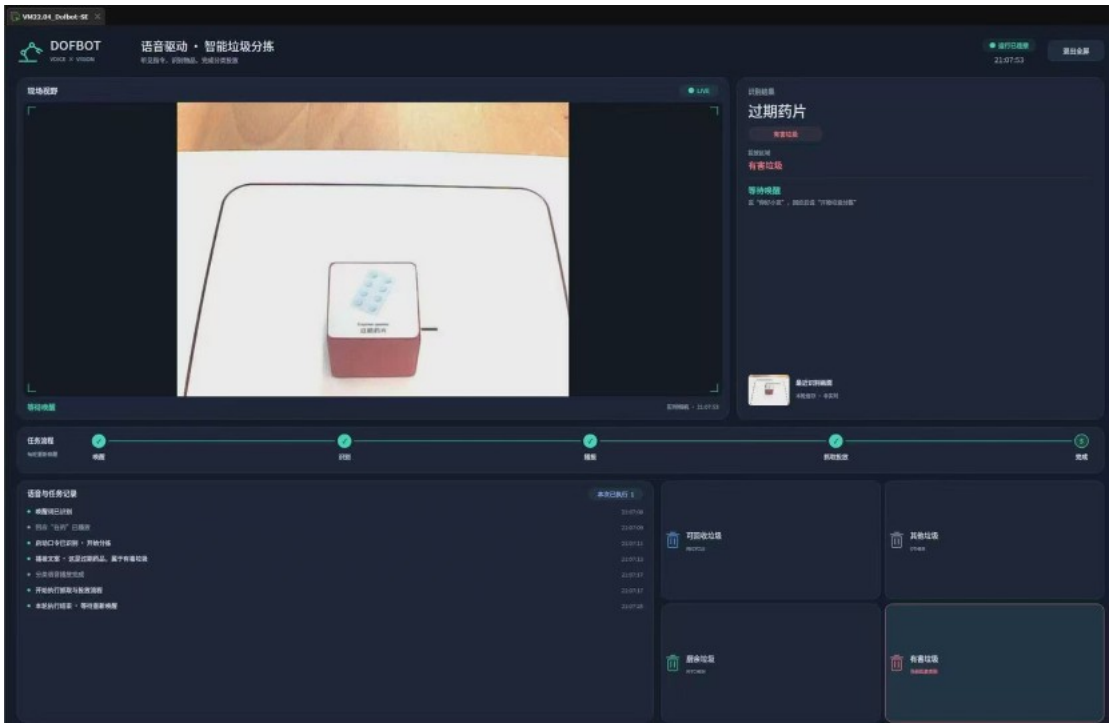


图 15　固定工位分拣软件界面：过期药片 → 有害垃圾的一轮任务

图 15 是“过期药片 → 有害垃圾”的一轮：现场视野中是样本盒，识别结果与投放类别均为有害垃圾，任务流程五段全部点亮，右下角的“有害垃

圾”投放区高亮；语音与任务记录按时间列出 7 条事件，从唤醒、回应、启动、播报文案到本轮结束，时间戳连续。界面右上角的“本次已执行”计数与“最近识别画面”让人能判断这是本轮结果，而不是上一轮的残留。界面上的投放区沿用“厨余垃圾 / 其他垃圾”的旧称，与垫面的湿垃圾、干垃圾一一对应。

3.3 特性成果

按功能项逐条给出可观察的结果（表 14）。功能验证以现场记录与离线检查为主：现场记录说明系统在真实设备上完成了动作，离线检查说明模块在固定输入下的行为可重复。

表 14 功能项验收

功能项	观察结果	证据类型
语音唤醒与启动	“你好小亚”回应“在的”，“开始垃圾分拣”进入任务	现场
稳定识别与播报	连续 3 帧稳定后播报物品与投放类别	现场
抓取投放	播报完成后执行对应类别的固定工位动作	现场，3 次抓取回调
只读展示	画面、类别、阶段、记录与投放区同步刷新	现场截图（图 15）
识别超时处理	未找到稳定目标时本轮结束，不进入抓取	现场，1 次超时记录
坐标播报	移动物块后播报 30.5 cm 与 19.2 cm，新请求仅播报一次	现场个例

功能项	观察结果	证据类型
候选一致性	合成序列基线帧 10 后提交 11/12/13，恰消费一次许可	离线回放
寄存器一致性	ID、版本与字段与 registers.json 一致，非法访问返回 SLVERR	RTL 仿真

3.3.1 优化过程与前后对比

优化分三层：处理分辨率与传输、默认语义分支的冗余计算、资源受限下的算子结构。原独立 Z19 配置中，逐像素 PIO 与 PS 侧处理共同决定发布延迟；balanced 配置保留 1280×720 的采集视野，把进入 PL 的处理尺寸降为 640×360，并配合原生 32 位访问与符合端点容量的传输方式，使发布率与端到端耗时同时改善（表 15、图 16）。

表 15 优化前后的性能与资源对照

优化项	优化前	优化后	判据与说明
视频处理像素量	1280×720（92.2 万像素/帧）	640×360（23.0 万像素/帧）	发布率 1.924 → 7.26 fps；采集至发布 567.99 → 约 187 ms
PIO 处理记录	472.64 ms	116.78 ms	同路径、不同像素量的采样
FIR 乘加结构	按 63 个乘法器并行设计	1 个乘法器按 tap 时分复用	每样本 6,250 周期预算；集成实现 DSP 14（0.71%）

优化项	优化前	优化后	判据与说明
默认语义分支	每帧复制/清零颜色掩码并做连通域	直接返回 not_requested	源码级删除无关计算；未单独计时
顶层资源（范围不同，仅列示）	原颜色链 1,885 LUT / 1,936 FF / 6 DSP	四功能集成 7,292 LUT / 8,882 FF / 14 DSP / 0 BRAM	两者统计范围不同，不计节省比例

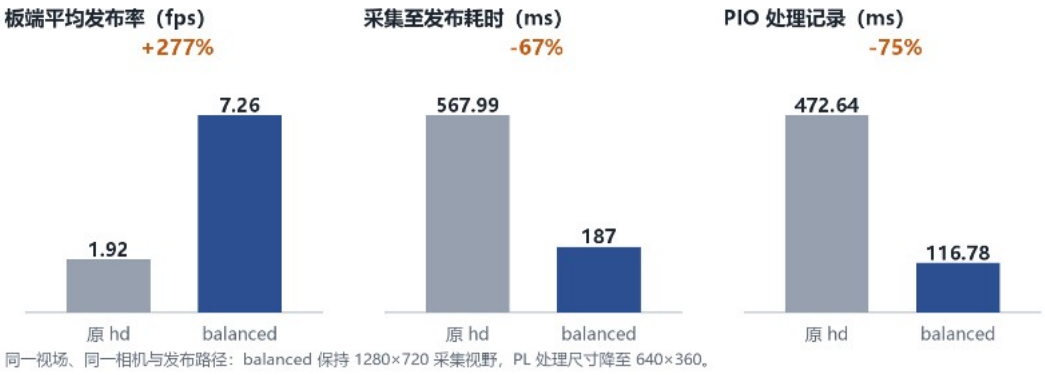


图 16　优化前后对比：发布率、采集至发布耗时与 PIO 处理记录

两组数据的处理分辨率不同，因此这组对比描述的是“在保留原始视野的前提下，把进入 PL 的像素量降到四分之一”这一系统取舍，而不是相同画质下的算法加速比。取舍的代价是小目标细节减少：处理尺寸减半后，小物体的边缘在阈值统计中更容易被切碎，这也是后续按物料调整 ROI 与面积区间的原因。

结构优化同样围绕瓶颈展开：默认语义分支不再执行仅供颜色关联使用的掩码复制与连通区域分析，省去与语义许可无关的工作；ROI 计数、包围框与

一阶矩在同一次像素遍历中完成，避免为每个统计量重复扫描；63 抽头 FIR 由全并行改为单乘法器时分复用，以采样吞吐换资源；候选段由相邻帧比较改为固定锚点比较，把缓慢漂移从“每步合格”变为“整段受约束”。

四功能集成工程随后在同一目标频点完成收敛：100 MHz 下 WNS +0.632 ns、WHS +0.010 ns，新增视频、音频与门控三个 PL 端点后，原定 100 MHz 目标仍然满足，为后续引入 DMA、更高分辨率或更多算子保留了裕量。

3.3.2 资源与时序

三份资源报告来自同一器件与同一工具版本（表 16）。颜色链与四功能集成工程为完整实现的布局布线结果，候选门控为独立 OOC 综合结果；三者顶层与统计范围不同，分别对应各自的模块，不相加为新系统的资源占用。

表 16 资源与时序

工程	报告阶段	LUT	FF	BRAM	DSP	WNS / WHS
原颜色链	完整实现（布局布线）	1,885	1,936	0	6	4.580 / 0.017 ns
候选门控	OOC 综合	866	949	0	0	7.112 / 0.061 ns
四功能集成	完整实现 + bitstream	7,292	8,882	0	14	+0.632 / +0.010 ns

四功能集成工程的时序与资源来自完整实现的布局布线报告：27,352 个端点中 0 个失败，按 WNS 推算的等效 Fmax 约 106.7 MHz；DRC 为 0 Critical、0 Error，52 条 Warning 全部属于 DSP 输入/输出流水建议与 1 条无负载信号实现后 LUT 占器件的 1.40%、DSP 占 0.71%、BRAM 为 0，说明三个 PL 端点与门控共用同一器件仍有充足余量。Vivado vectorless 估算片上功耗 4.167 W（动态 2.955 W、静态 1.212 W），该数字为工具估算，不是板上实测。

3.3.3 测试与验证记录

验证按“仿真—综合与实现—现场”组织（表 17）。仿真检查定点算术与总线行为，综合与实现检查资源、时序、DRC 与位流生成，现场记录检查整机是否按设计完成一次任务。各级记录都保留输入、版本与原始输出，便于复现与比对。

表 17 测试与验证记录

检查项	覆盖内容	结果
HSV 数值仿真	1,024 组 RGB 向量、数值容差、有效间隙与 TLAST	HSV_PASS
视频流仿真	2,064 个数据拍，背压、SOF/EOL、复位与帧内配置快照	COLOR_AXIS_PASS

检查项	覆盖内容	结果
AXI-Lite 仿真	64 像素、四种颜色配置、AW/W 解耦、读取停顿与错误恢复	COLOR_AXI_PASS
候选门控 RTL	661 次断言检查、418 次写、66 次读；超时优先级与复位	SORT_GUARD_PASS
机械臂软件	65 项状态机/启动/预览/展示 + 4 项协议 + 9 项安装	78 项通过
候选适配软件	接口模型、元数据校验、寄存器模型与离线回放	94 项通过
网格留出点	两组样本均值/RMS/最大：1.03/1.24/2.16 mm、1.68/1.96/3.05 mm	图内检查残差
MIPI 相机预览	1920×1080，757 帧 / 30.047 s	25.16 fps
四功能集成构建	综合、实现、时序收敛与 bitstream 生成，0/27,352 失败端点	WNS +0.632 ns

接口一致性检查贯穿上述过程：同一份 registers.json 生成 RTL 头文件、C 头文件与 Python 定义，任何一处字段改动都会在三端同时体现；模型清单把权重、类序与框架来源固定下来，加载与入册共用同一套校验；配置示例与文档逐项对应，避免“文档写一套、代码跑另一套”。

3.4 大模型协作记录

大模型参与的是开发过程，不是产品运行时的推理：YOLO 与 Vosk 仍在

PS 的 CPU 上运行，PL 不含大模型。本节记录的提示词、输出与修正都可以在 Git 提交里复核。协作方式是“单一代码助手 + 人工审计”的闭环（图 17）：用户给出任务范围与执行边界，助手生成工程，用户或外部审计指出语义矛盾，交叉源码审阅定位，集中修订并保留提交，共性判断再沉淀为技能。



图 17　开发协作闭环：任务约束、生成、审计、修订与沉淀

真实任务来自用户的《Codex 总执行提示词：Z19 PS/PL 四功能工程补充与源码交付》（2026-10-30）。提示词明确四项源码任务（视频预处理、垃圾识别、定位与候选、语音前端）与执行边界：本轮只允许源码、文本与接口审阅，不运行仿真、综合、推理或设备验证。这条边界本身就是工程约束——它把“交付了什么”与“验证了什么”分开记录，避免把静态检查写成运行结论。

表 18　四次纠错与一次遗漏的实际轨迹

案例	初次输出的问题	审计与修订	提交
语义候选与颜色约束	候选许可统一放在 association.state == associated 之后；无指定颜色时，唯一语义目标也无法进入许可	用户审计：“现有方案不适合未经场景约束就作为全部 16 种垃圾的统一抓取门槛”；改为默认 semantic_bbox	8a76e84
口令与跨任务语音	采用“小助手”唤醒；只清接收端队列，忙碌期半句可能在本轮结束后才完成 ASR	统一“你好小亚”与“开始垃圾分拣”；新增 /voice/boundary，重建采音—滤波—ASR 代际	8a76e84
新入口只读观测	继承任务主流程但漏接 save / recognized 回调；编码期间换帧会让旧图配上新身份	用户审计：“它是新增协同方案的软件完整性问题”；接回回调，图像与来源同快照发布	8a76e84
模型来源与目标环境	内层尺寸、stride、IoU、max_det 与 letterbox 为假设值，来源与目标环境未分开	用户审计：“这些信息本轮没有取得”；未知参数保留 null，入册与加载共用同一清单	8a76e84
安装来源与测试契约	只补了冻结检出说明，未同步安装器与测试的默认源，两个 Dashboard 文件与原清单哈希冲突	助手承认：“上轮我只补了冻结检出的说明，没有同步解决安装源与测试源，这是遗漏”；冻结 15 文件 payload	2176fbc

这五次修订都是人工审计先指出“这与任务语义不符”，再由提交把修订固定下来。由此形成的分工是：助手负责跨文件实现、接口对应与缓存链追踪人负责确认场景、证据归属与授权范围；提交负责让结论可复核。产品没有部署运行时大语言模型，技能与文档也不转述为识别率或响应时间。

3.5 技能包的提炼过程

技能不是“总结几条原则”，而是把一次具体失败抽成可执行的检查流程。四份技能采用同一结构：输入与输出、逐步检查、可改参数、失效条件、检查建议，最后一张“经验与证据对应”表把每一步连回原始记录。提炼分五步：发现问题、定位共同原因、形成可执行流程、核对有效依据、确定复用边界。

表 19　四份技能的来源、验证与边界

技能	从什么失败或观察提炼	验证依据	失效边界
像素流与帧契约	背压时标签与算术不同步；配置跨帧被改写；统计漏掉末像素	旧颜色流 COLOR_AXIS_PASS 的背压/SOF/EOL/复位；新帧快照与 ACK 的源码	标签延迟未知、跨时钟域、未消费完整帧时不得判定有效
寄存器与 PIO 事务契约	AW/W 先后、响应停顿、非法与部分写；地址定义分散在多处	旧门控 661 次断言中的事务检查；registers.json 的三端投影	不提供 DMA、缓存一致性或未知地址探测
语音任务边界	忙碌期半句口令在本轮结束后被补成新的唤醒	原状态机的单轮任务与忙碌隔离；新增 boundary_id 重建链	不保证开放词表、降噪效果或跨源混用
控制与只读观测	新入口继承控制却漏接回调；旧图配新身份	原界面的设备所有权与陈旧画面拒绝；新预览的同快照发布	显示 LIVE 不等于物理抓取成功

有效性来自两侧：旧记录证明经验来源真实，源码修订证明流程能落到具体位置；两者都不等于“技能已在他队验证”。边界写在失效条件里——像素

流技能不解决 DMA 与跨时钟域，寄存器技能不提供未知地址探测，语音技能不保证开放词表。技能随仓库以文档交付，使用时把目标工程、技能文件与任务范围一起交给代码助手；缺文件或含义未确认时返回具体缺失项，而不是用猜测值补成结果。

3.6 从零复现说明

以下步骤面向“拿到仓库、没有现场记录”的使用者。两条路径分别对应现用固定工位演示与四功能集成工程，环境、镜像与产物互不替代。

3.6.1 准备环境

固定工位演示需要配套 Ubuntu 22.04 + ROS 2 Humble 桌面、DOFBOT SDK 与识别模型、中文播报音频与逆解服务；四功能工程需要 Vivado 2020.1（保留 ALINX 的 PS/IP 配置）、匹配的 AArch64 sysroot 与板端 Linux，以及合法取得的权重、类序与 Vosk 词典。

3.6.2 取源码并固定版本

```
git clone https://github.com/Citlali6/VisionGrasp-Z19.git
cd VisionGrasp-Z19 && git rev-parse HEAD
```

把实际 SHA 与材料版本记录在一起；本次构建对应源码提交 4e51e5b。不

要用会持续移动的分支名代替版本。

3.6.3 复现固定工位演示

```
bash robot/dofbot/install.sh --check
bash robot/dofbot/install.sh
bash ~/启动垃圾分拣.sh
bash ~/停止垃圾分拣.sh
```

预检核对文件哈希、依赖与运行状态；安装只写应用文件与启动脚本，不启动控制、不打开串口、不加载 FPGA。启动后把物块放到样本区，先说“你好小亚”，听到“在的”再说“开始垃圾分拣”；运行记录位于安装目录的 records 子目录。

3.6.4 构建四功能工程与传输库

```
vivado -mode batch -source fpga/system_pspl/create_project.tcl \
  -tclargs /outside-source/pspl
vivado -mode batch -source fpga/system_pspl/build_bitstream.tcl \
  -tclargs /outside-source/pspl/project/z19_pspl_four_functions.xpr \
           /outside-source/pspl/system_pspl.xsa
cmake -S host/libpspl -B /outside-source/pspl-c \
  -DCMAKE_TOOLCHAIN_FILE=$PWD/host/libpspl/aarch64-linux.cmake
cmake --build /outside-source/pspl-c
```

输出目录必须不存在，脚本拒绝覆盖；Windows 下构建路径用纯 ASCII，Vivado 2020.1 在中文路径上会读文件失败。完成后保存 utilization、timing、DRC 报告以及 bit/XSA 的哈希。

3.6.5 模型入册与私有配置

```
python3 -m host.app.perception.enroll_model \
  --weights /legal/best.pt --framework-root /legal/ultralytics \
  --names-yaml /legal/data.yaml \
  --source-contract /private/source_contract.json \
  --output /private/model_manifest.json
```

按配置示例补齐相机 by-id、UIO 节点、候选策略、语音源、绑定地址与 token（至少 32 字符，只放私有文件）。入册只形成加载契约，不表示已在板端完成推理。

3.6.6 启动与保存证据

```
python3 -m host.app.perception.service --config /private/pspl-service.json
/usr/bin/python3 -B /absolute/path/robot/dofbot/pspl_entry.py \
  --config /private/vm-pspl.json
```

先不带 --execute 观察候选与播报，确认数据链后再显式启用动作。应保存的产物与检查项见表 20。

表 20 复现产物与验收项

应保存的产物	验收时检查什么
bootstrap/status 与会话目录	PID、start ticks、boot ID、运行 UUID 与 generation 是否一致
cycle-NNN-detection.jpg/.json	图像、来源帧号、配置、类别与许可是否同帧对应
preview/latest.jpg 与 live.json	原始采集标记、实际新帧与新鲜度；不得当作 PL 效果或模型输入
工具报告与 bit/XSA	器件、工具版本、报告阶段与 SHA-256 一并记录

出现以下情况应停止并保留错误：缺权重或类序不符、UIO 身份或 map 尺寸不符、边界确认超时、相机断流或帧号不连续。复现的目的是得到可核对的记录，而不是绕过失败。

第四部分　总结

4.1　后续发展方向

针对未来发展方向，我们可围绕现有不足的深化优化与场景拓展展开。

数据与模型层面：面向垃圾站、园区与校园等不同来料构建含特定干扰的定制数据集，用合成样本补充真实数据；测试轻量检测与分割模型并结合蒸馏在 AArch64 上实测精度与帧率，平衡识别率与延迟。

资源与功耗层面：以 7,292 LUT、14 DSP 的实测结果为基线建立资源预测，按图像复杂度动态调度（调整处理尺寸或跳帧）；把逐像素 PIO 换成 DMA 与缓存一致性传输，探索 PL 动态时钟，用板上实测功耗替代 vectorless 估算。

机械臂与控制：完成手眼标定与夹爪偏移补偿，把固定拾取扩展为平面内任意位置抓取；引入视觉伺服与抓取反馈，动作先在仿真中验证再迁移到实物避免夹持失败与打滑。

定点与量化：在现有位宽反推的基础上做混合精度与按层位宽分配，结合资源约束尝试量化感知训练，在精度损失可控的前提下继续压缩 DSP 与布线。

场景拓展：支持更多类别与容器，按来料节拍组织多工位并行与容器更换；把任务、候选与结果回传形成运营看板，逐步从单台工位走向可复制、可计量的节点。

4.2 心得体会

这项工作让我们修正了对“智能分拣”的直觉。最初我们以为只要模型识别得准，系统就能完成分拣；真正联调之后才发现，决定成败的是识别之后的那几步：这一帧是否足够新，目标是否仍在工位内，位置是否已经漂移，这次许可有没有被消费。把这些条件写进 RTL 与状态机，比在演示脚本里加判断更难，但它让系统的行为可以被解释、被复现。后来我们把场景从展台移到垃圾站的分拣位：在那里，一次识别如果止步于屏幕，就仍然只是演示。

软硬件划分也让我们改变了做法。划分不是“能放到硬件就放到硬件”：逐像素、逐样本的乘加适合定点流水；模型框架、词典与坐标配置依赖成熟的 Linux 生态，留在 PS 更稳；机械动作依赖厂家 SDK 与 ROS 环境，放在上位机可以复用已经验证过的轨迹。我们最后用三个标准来判断：数据的规律性、依赖的可获得性、失败的可观察性。凡是失败方式说不清的计算，宁可不放进

硬件。

验证方法上的收获更大。一次综合报告如果不说清器件、工具版本与报告阶段，就不能用来支撑“优化”；一次现场演示如果不记录帧号与配置，就不能说明问题出在算法还是接口。我们因此为每个模块保留仿真标记、资源报告与板端日志，并把性能口径写清楚——发布率不是 PL 吞吐，采集至发布包含解码、传输与编码。这个习惯在写材料时同样有用：它让人不容易把设计参数当成实测结果。

在协作方式上，大模型帮我们补齐了跨文件接口、寄存器投影与大量文档工作，也留下过真实的错误：把颜色变成所有类别的共同门槛，漏接新入口的展示回调，只清理接收端而忽略生产端的语音缓存。每一次都是人先发现“这与任务语义不符”，再由审计与提交记录把修订固定下来。我们由此形成了一个判断：AI 可以加快实现，但不能代替对任务语义的把握和对证据的核对。

最后是团队协作的体会。分拣看起来是一个演示，实际涉及视觉、语音、硬件、机械与文档五个方向，任何一处口径不一致，整机就会在最薄弱的环节上停下来。我们逐渐习惯先统一接口与单位，再各自实现；先约定失败时的表

现，再讨论成功时的效果。作品的价值也正在这里：它把识别、候选、交互与执行组织成一条有记录、可解释、能继续改进的工程路径。

第五部分　参考文献

[1] AMD (Xilinx). Zynq UltraScale+ Device Technical Reference Manual: UG1085 [Z]. 2023.

[2] AMD (Xilinx). Vivado Design Suite User Guide: Synthesis: UG901 [Z]. 2020.

[3] ARM. AMBA AXI and ACE Protocol Specification: ARM IHI 0022 [S]. 2021.

[4] ALINX. Z19 开发板硬件用户手册与原理图 [Z]. 2022.

[5] Ultralytics. YOLO11 Documentation [EB/OL]. https://docs.ultralytics.com/models/yolo11/, 2025.

[6] Alpha Cephei. Vosk Speech Recognition Toolkit [EB/OL]. https://alphacephei.com/vosk/, 2025.

[7] LAPEREX. cv_rgb_to_hsv: RGB to HSV conversion in SystemVerilog (MIT License) [EB/OL]. https://github.com/laperex, 2024.

[8] Yahboom. DOFBOT SE 机械臂与视觉识别教程 [EB/OL]. https://www.yahboom.net/study/DOFBOT_SE/, 2024.

[9] OpenCV. Camera Calibration and 3D Reconstruction [EB/OL]. https://docs.opencv.org/, 2025.

[10] SMITH S W. The Scientist and Engineer's Guide to Digital Signal Processing [M]. 2nd ed. San Diego: California Technical Publishing, 1997.

[11] BLACKMAN R B, TUKEY J W. The Measurement of Power Spectra [M]. New York: Dover Publications, 1958.

[12] 国家市场监督管理总局, 国家标准化管理委员会. GB/T 19095—2019 生活垃圾分类标志 [S]. 北京: 中国标准出版社, 2019.

[13] 国家质量监督检验检疫总局, 中国国家标准化管理委员会. GB/T 25180—2010 生活垃圾综合处理与资源利用技术要求 [S]. 北京: 中国标准出版社, 2010.

[14] 中华人民共和国住房和城乡建设部. CJJ 90—2009 生活垃圾焚烧处理工程技术规范 [S]. 北京: 中国建筑工业出版社, 2009.

[15] TOMRA. AUTOSORT: sensor-based sorting for waste and metal recycling [EB/OL]. https://www.tomra.com/waste-metal-recycling/products/machines/autosort, 2025.

[16] AMP Robotics. AMP ONE and AMP Cortex sorting systems [EB/OL]. https://ampsortation.com/technologies, 2025.

[17] SUTCO. Pre-sorting before power plant / incineration [EB/OL]. https://www.sutco.com/, 2025.

[18] European Institute of Innovation and Technology. ZRR municipal waste smart sorting robot [EB/OL]. https://www.eit.europa.eu/, 2024.

[19] Health and Safety Executive (UK). Health risks in the waste and recycling industry [EB/OL]. https://www.hse.gov.uk/waste/health.htm, 2024.

[20] Citlali6. VisionGrasp-Z19: 基于 Zynq 软硬件协同的语音视觉垃圾分拣系统 [EB/OL]. https://github.com/Citlali6/VisionGrasp-Z19, 2026.